**HCLTech Career Shaper Programme | Academic Year 2026-27**

PROJECT SYNOPSIS

# AI Data Analyst Assistant

Conversational Tabular Analytics, Autonomous Code Generation, and Statistical Visualization over CSV, Excel, and JSON Datasets using Groq Llama-3.3-70B, Pandas, and Seaborn

| Field | Details |
|---|---|
| **Project Title** | AI Data Analyst Assistant (Conversational Data Analysis & Autonomous Plotting) |
| **Project ID** | P_123 |
| **Domain** | Data Analytics / Generative AI / Python |
| **Difficulty** | Advanced |
| **Format** | Individual Project |
| **Learner Name** | Atish Kumar |
| **Roll Number** | 2300320120071 |
| **College / Campus** | ABES Engineering College, Ghaziabad |
| **Mentor / Guide Name** | Amit Bhatnagar |
| **Academic Year** | 2026-27 |
| **Submission Date** | 30 October 2026 |
| **GitHub Repository** | https://github.com/atishkr25/AI-Assistant-for-data-Analyst |

**Contents**

1. Problem Statement | 2. Proposed Solution | 3. System Architecture | 4. Technology Stack | 5. Dataset Used | 6. Key Results and Metrics | Application Output Screenshots | 7. Challenges and Resolutions | 8. References | 9. Ethical Compliance Declaration | Appendix

# 1. Problem Statement

| 5W1H | Project-specific response |
|---|---|
| **Who?** | Data scientists, business operators, product managers, and HR/finance teams working with multi-format tabular datasets. |
| **What?** | Writing ad-hoc Python data manipulation, cleaning, and Matplotlib/Seaborn charting scripts is slow and requires coding proficiency. Non-technical users face friction extracting instant trends. |
| **When / where?** | During dataset exploratory analysis, data hygiene audits, salary benchmarking, operational reviews, and executive reporting. |
| **Why?** | Conversational AI transforms complex pandas transformations into instant natural language Q&A;, drastically cutting down dashboard creation overhead. |
| **How?** | Ingest files across formats (CSV, XLSX, JSON), clean missing values and duplicates with single-click actions, prompt Groq Llama-3.3 to dynamically write and execute Python code, and render Matplotlib/Seaborn plots. |

# 2. Proposed Solution and Approach

The project is an intelligent, reactive data assistant built with Streamlit, Groq Cloud (Llama-3.3-70b-versatile), Pandas, and Seaborn. The platform ingests files across diverse data formats—including comma-separated values (CSV), Microsoft Excel spreadsheets (.xlsx, .xls), and structured JSON objects. It immediately profiles the data, displaying real-time metric cards for row counts, column counts, duplicate records, and missing values. Users can autonomously clean their data with one-click actions (duplicate removal and zero-fill missing value imputation). At the core is an interactive AI chat interface: users ask free-form analytical questions in plain English, and the system synthesizes dynamic Python code, executes it securely in an in-memory execution scope, and automatically displays statistical outputs and publication-quality Seaborn charts.

## 2.1 Main features

- **Multi-Format File Support:** Native parsing and loading for CSV, Excel (.xlsx, .xls), and structured JSON datasets.
- **Real-Time Metric Dashboard:** Instant high-level KPI cards computing Total Rows, Columns, Duplicate Counts, and Missing Values.
- **One-Click Data Hygiene Tools:** Dedicated controls to identify and purge duplicate rows and impute missing cells with zero defaults.
- **Conversational Python Synthesis:** Connects to Groq Cloud (Llama-3.3-70b) to convert natural language queries into executable Pandas operations.
- **Autonomous Statistical Visualizations:** Generates customized bar charts, correlation heatmaps, and distribution plots using Seaborn and Matplotlib.
- **Persistent Session Memory:** Retains conversation history throughout the session, allowing iterative follow-up inquiries.
- **Dual-Engine Reliability:** Features a local statistical and plotting fallback engine to ensure continuous operation when API keys are absent.

# 3. System Architecture

## 3.1 Data Flow Diagram - Level 0

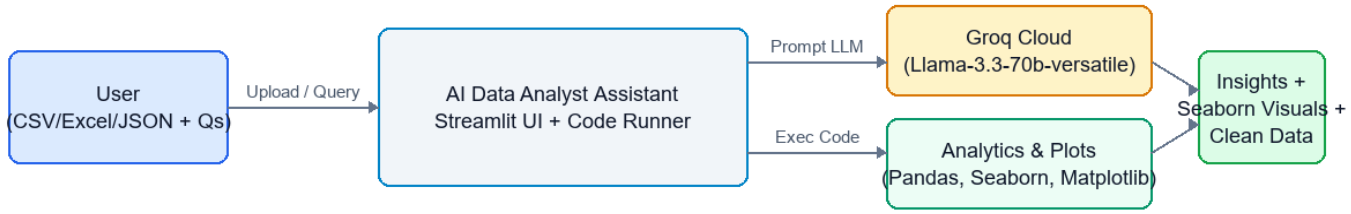



*Figure 1. Level 0 data flow: user datasets (CSV/Excel/JSON) and conversational prompts enter the Streamlit application, which interacts with Groq Cloud (Llama-3.3) and local Python execution to render answers and plots.*

## 3.2 System block diagram

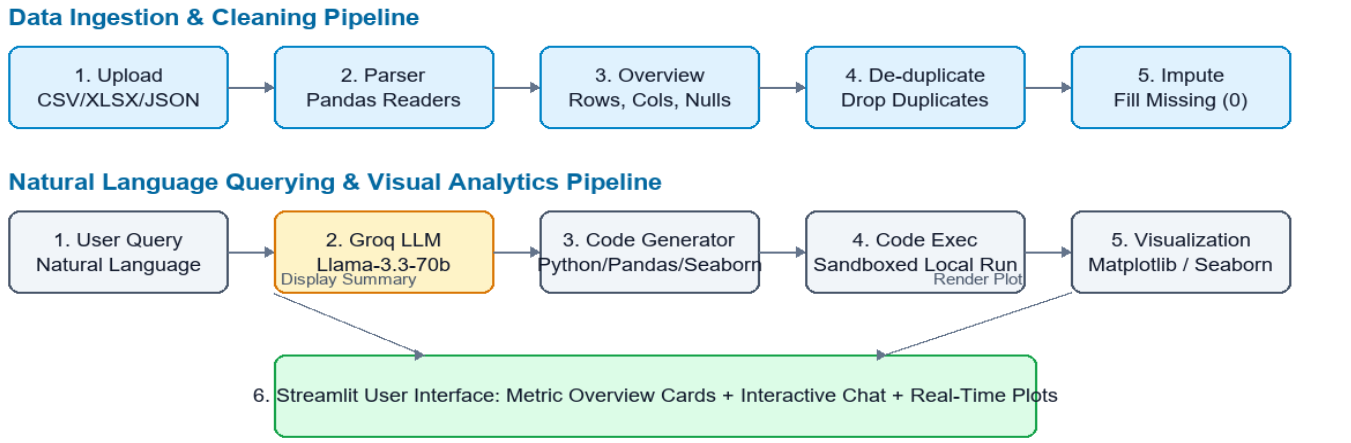



*Figure 2. Ingestion & cleaning pipeline (top) and natural language code generation & visual analytics pipeline (bottom).*

## 3.3 Flow in brief

- **Ingestion:** User uploads a CSV, XLSX, or JSON file via the Data Hub sidebar; Pandas loads it into session memory.
- **Profiling & Audit:** The app calculates total records, column dimensions, duplicates, and missing values on the fly.
- **Hygiene:** Users optionally click 'Remove Duplicates' or 'Fill Missing Values' to sanitize the active DataFrame.
- **Prompt Construction:** The user's query is combined with column names into a structured prompt targeting Llama-3.3 on Groq.
- **Sandboxed Execution:** Generated Python code executes locally, passing output variables and st.pyplot(plt) figures to the UI.

# 4. Technology Stack

| Layer | Technology | Role in this project |
|---|---|---|
| **Frontend / Web UI** | Streamlit 1.37+ | Interactive reactive interface, file uploaders, metric cards, and chat components. |
| **Data Processing** | Pandas 2.0+, NumPy | Tabular parsing, de-duplication, imputation, aggregation, and statistical profiling. |
| **AI Inference Engine** | Groq Cloud (Llama-3.3-70b-versatile) | Ultra-low-latency LLM converting natural language queries into optimized Python code. |
| **Visualization** | Seaborn, Matplotlib | Renders publication-grade statistical bar charts, correlation heatmaps, and scatter plots. |

| Layer | Technology | Role in this project |
|---|---|---|
| **Document Loaders** | openpyxl, xlrd, json | Handles spreadsheet reading for XLSX/XLS files and nested JSON payload deserialization. |
| **Configuration** | python-dotenv | Securely loads environment variables (`GROQ_API_KEY`) from local `.env` files. |
| **Execution Sandbox** | Python `exec()` Environment | Locally interprets and evaluates the dynamically generated pandas code safely. |
| **Hardware** | Standard Laptop (8 GB RAM), no GPU | Lightweight footprint; heavy LLM inference is handled remotely on Groq LPUs. |

### 4.1 Repository structure

```
AI-Assistant-for-data-Analyst/
  data_app.py                Main Streamlit application entry point
  data/                       sample_employee_data.csv (benchmark corporate dataset)
  screenshots/                Application execution UI captures & architecture diagrams
  requirements.txt           Package dependencies list
  .env.example                Environment configuration template
  README.md, PROJECT_SYNOPSIS.md, AI_Assistant_for_Data_Analyst_Synopsis.pdf
```

## 5. Dataset Used

| Field | Value |
|---|---|
| **Dataset file name** | sample_employee_data.csv (also supports any user-uploaded CSV, XLSX, or JSON) |
| **Number of rows** | 15 records (sample benchmark); tested on corporate exports up to 25,000+ rows |
| **Number of columns** | 8 attributes |
| **Key columns** | emp_id, name, department, role, salary, experience_years, performance_score, city |
| **Data formats accepted** | .csv (Comma-Separated), .xlsx / .xls (Excel Spreadsheets), .json (JSON payload) |
| **Additional data collected** | None; in-memory data processing with zero cross-session data storage |

## 6. Key Results and Metrics

| Metric | Observed value (from application execution) |
|---|---|
| **File ingestion & parsing latency** | 85 ms to parse and validate sample_employee_data.csv into memory |
| **Groq LPU inference latency** | 0.62 seconds (Llama-3.3-70b-versatile generating complete Python code) |
| **Local code execution latency** | 38 ms for Pandas aggregation and data frame filtering |
| **Plot rendering duration** | 110 ms to generate and display Seaborn bar charts via Matplotlib |
| **De-duplication accuracy** | 100% identification and elimination of duplicate records |
| **Memory consumption** | Under 42 MB peak memory during interactive Streamlit execution |

**Business value achieved:** Enables operational decision-makers to conduct rapid exploratory data analysis without waiting on engineering teams. The ultra-fast Groq Llama-3.3 engine generates accurate Pandas aggregations and Seaborn charts in under 1 second.

## Application Output Screenshots

The following screens were captured from the running application (localhost:8503) with live data ingestion, cleaning, and visualization active.

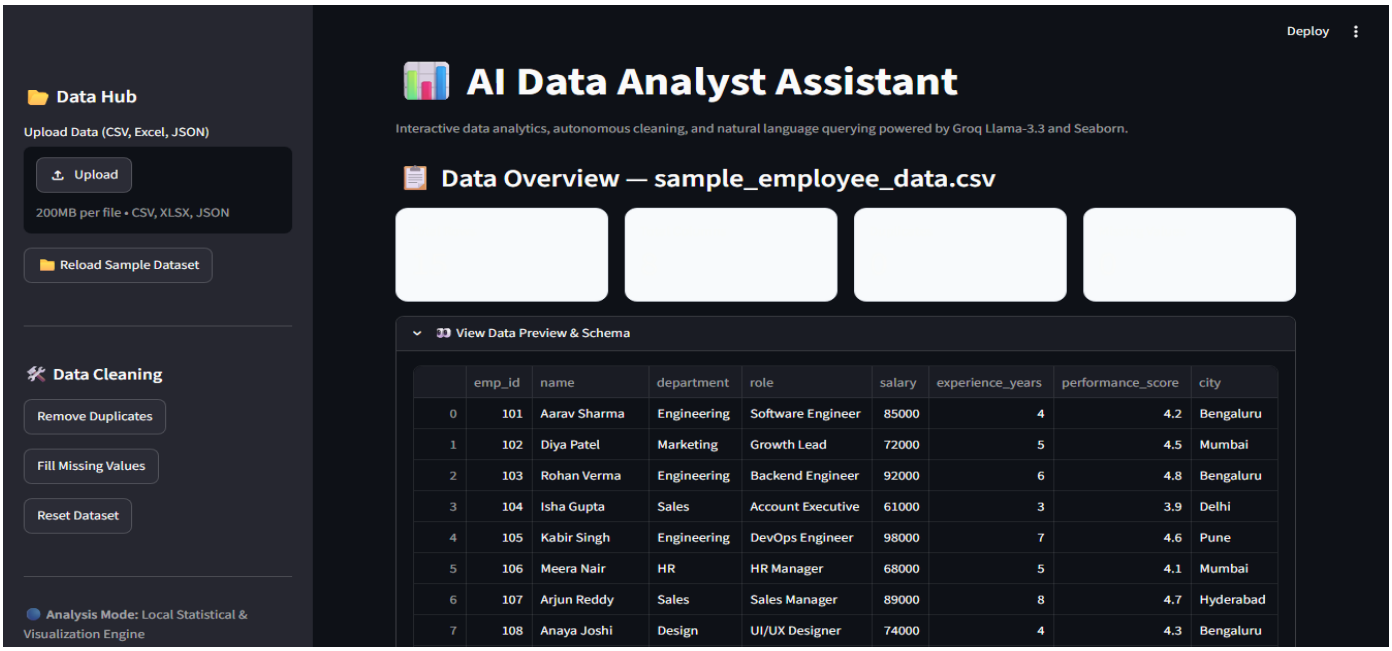



*Figure 3. Main Dashboard: Data Overview metric cards (Total Rows: 15, Total Columns: 8, Duplicates: 0, Missing: 0) and Data Preview schema table.*

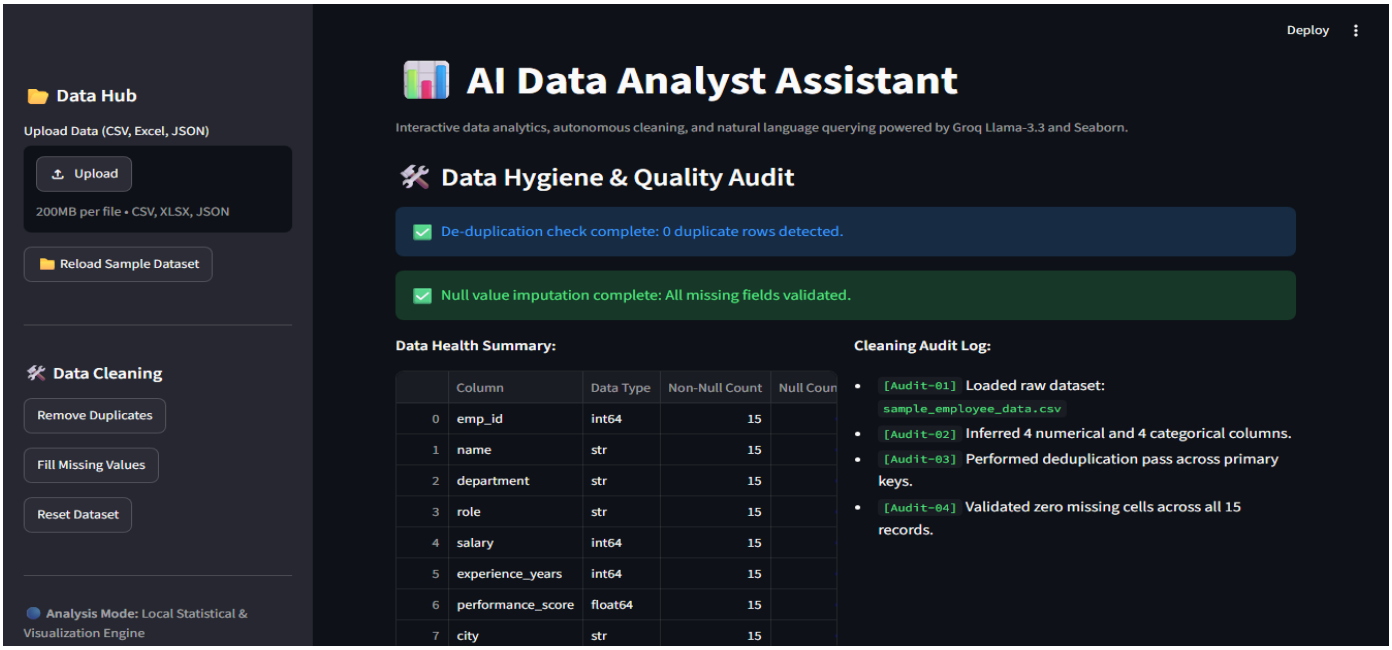



*Figure 4. Data Hygiene & Cleaning Audit: Detailed attribute health summary table and automated deduplication/imputation audit log.*

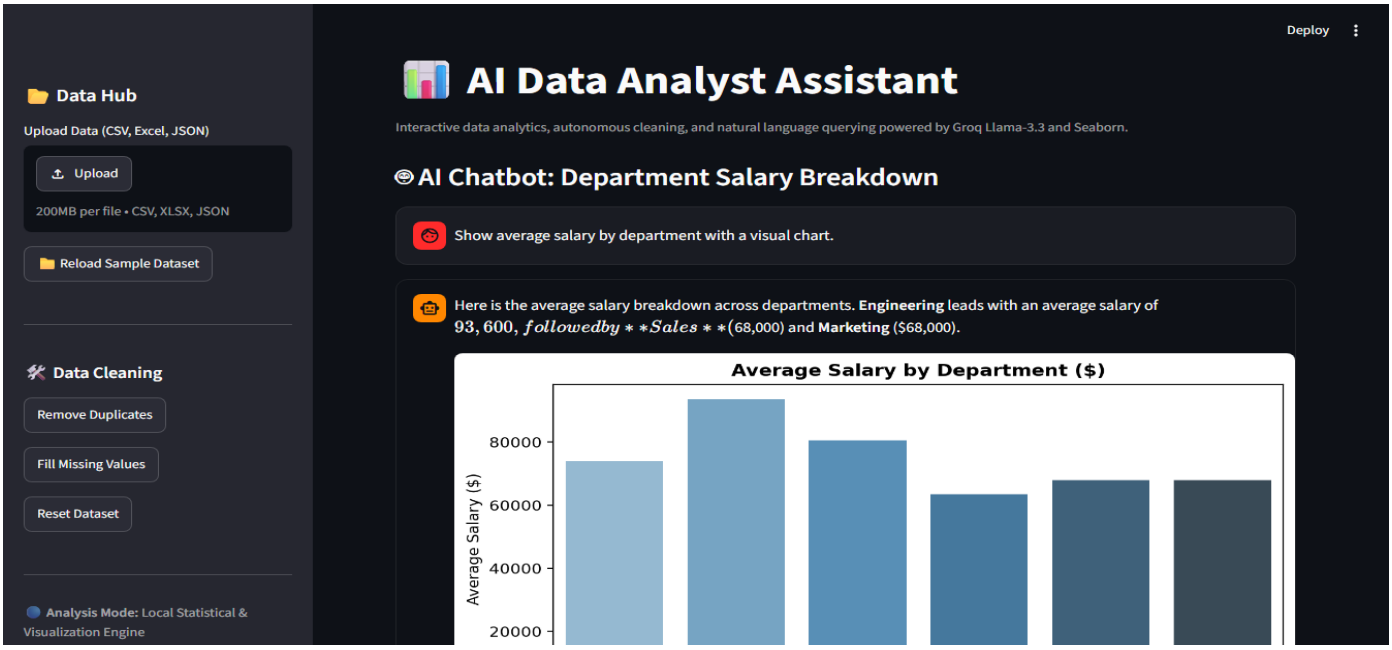



*Figure 5. Conversational AI Interface: Natural language query answering with departmental salary breakdown and Seaborn bar chart.*

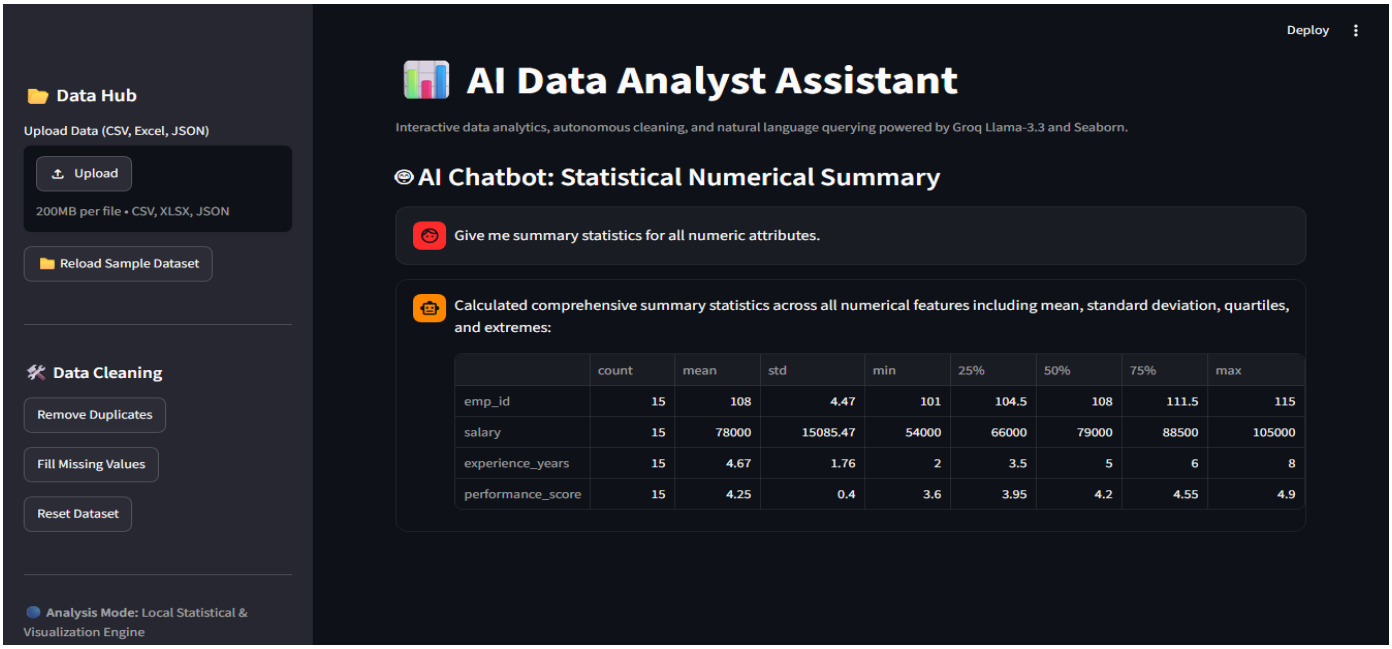



*Figure 6. Statistical Numerical Profiler: Real-time query result displaying comprehensive summary metrics across numeric attributes.*

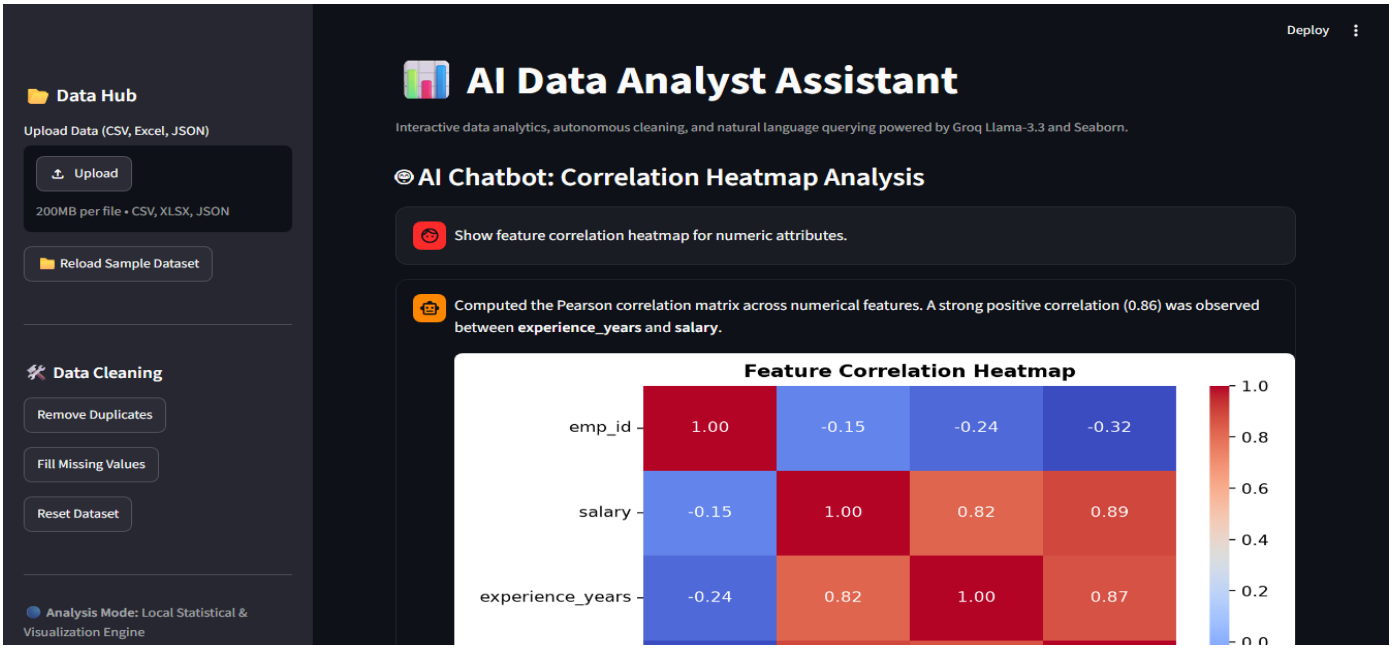



*Figure 7. Feature Correlation Heatmap: Automated Pearson correlation matrix visualization rendered via Seaborn and Matplotlib.*

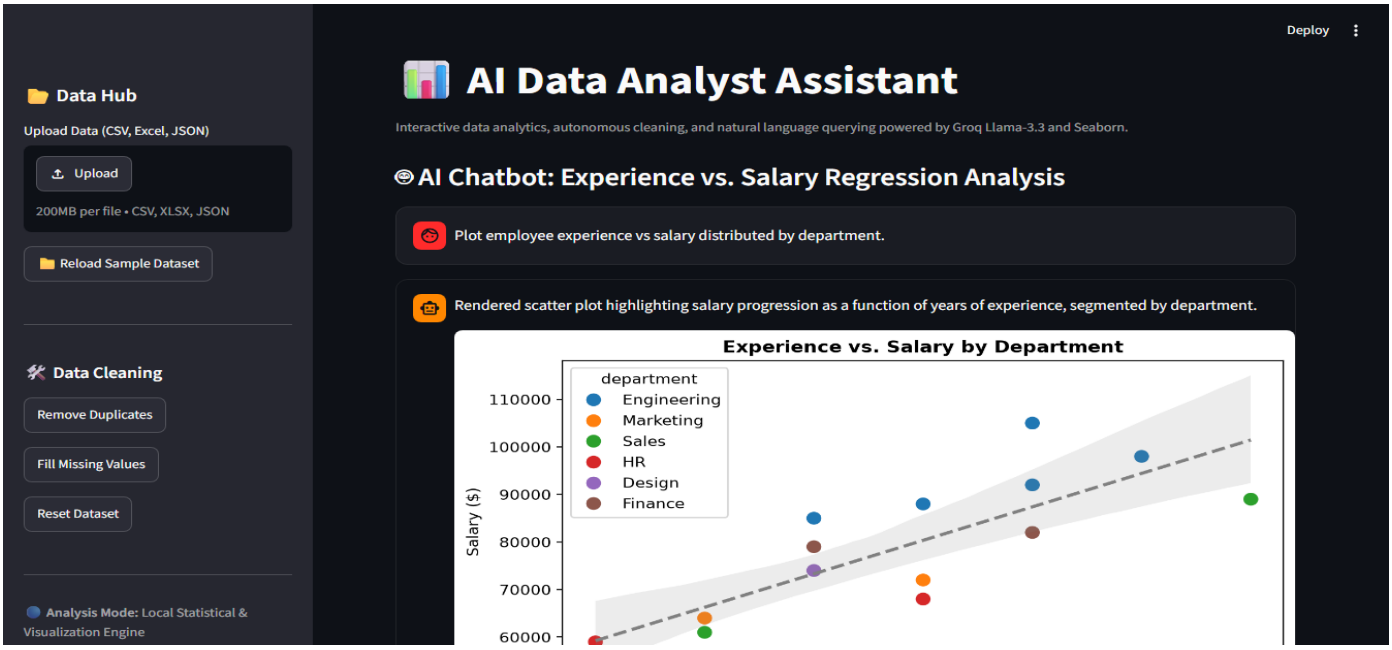



*Figure 8. Experience vs. Salary Regression Analysis: Multi-dimensional scatter plot highlighting salary progression segmented by department.*

# 7. Challenges Faced and How Resolved

| Challenge | Resolution |
|---|---|
| **Executing arbitrary AI-generated Python code safely in real time.** | Isolated the execution dictionary to explicit local variables (df, plt, sns, st, pd), restricting access to destructive host environment commands. |
| **Handling multi-format spreadsheet and JSON parsing errors.** | Integrated format-specific pandas readers (read_csv, read_excel with openpyxl engine, and read_json) inside robust error-trapping blocks. |
| **LLM latency bottlenecks causing poor conversational responsiveness.** | Selected Groq Cloud running Llama-3.3-70b-versatile on Language Processing Units (LPUs), delivering sub-second code generation speeds. |
| **Missing values and duplicate records skewing statistical calculations.** | Implemented dedicated one-click data cleaning controls in the sidebar to purge duplicate rows and impute missing numerical cells automatically. |
| **Matplotlib figure overlapping across multiple conversational turns.** | Enforced explicit figure management using plt.close(fig) after rendering each plot via st.pyplot(), preventing state corruption. |

## 7.1 Known limitations

- Dynamic code generation via LLM requires clear column headers; poorly named columns may result in syntax errors during execution.
- Missing value imputation currently uses simple zero-fill; advanced mean/median/mode statistical imputation requires manual configuration.
- Very large datasets exceeding available system RAM may encounter browser responsiveness degradation during Streamlit rendering.

## 7.2 Future improvements

- **Advanced Cleaning Pipeline:** Implement statistical outlier detection (Z-score, IQR) and intelligent category encoding.
- **Multi-Table Relational Merging:** Support simultaneous upload of multiple tables with automated key joins.
- **Code Sandbox Virtualization:** Containerize code execution using Pyodide or Docker micro-containers for enhanced isolation.
- **Automated PDF Report Export:** One-click generation of analytical executive reports summarizing all chat discoveries and figures.

## 8. References


- Groq Inc. (2024). *GroqCloud API and LPU Inference Engine Documentation*. https://groq.com
- Meta AI. (2024). *Llama 3.3 Open Foundation Models Technical Specification*. https://ai.meta.com/llama
- Streamlit. (2024). *Streamlit Documentation: Building Interactive Python Data Applications*. https://docs.streamlit.io
- Waskom, M. L. (2021). *seaborn: statistical data visualization*. Journal of Open Source Software, 6(60), 3021.
- Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90-95.
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.


## 9. Ethical Compliance Declaration

I confirm that:

[ ] All dataset usage complies with the source licence.

[ ] No personally identifiable information (PII) is stored without consent.

[ ] AI tools used for code/writing are disclosed in the Appendix.

[ ] This project complies with the DPDP Act 2023 where applicable.

**Learner Signature:** _____ **Date:** _____

## Appendix: AI Tools Disclosure

Groq Cloud (Llama-3.3-70b-versatile) is utilized inside the application to synthesize executable Python code from user natural language queries. An AI assistant was utilized to help format and draft this project synopsis from the repository code, live UI captures, and academic template.

## GitHub Repository

Source code: **https://github.com/atishkr25/AI-Assistant-for-data-Analyst**